

TERRAFORM - Condition-Operations

VISHWANATH M S
VISHWACLOUDLAB.COM

Agenda for the Day

- Conditioning in Terraform
- Iterations in Terraform
- Resources Metadata
- Data Source in Terraform
- Operators in Terraform
- Interpolation in Terraform

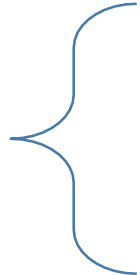
Conditioning in Terraform

Method 1



conditional

Method 2



Depends_o

Conditions in Terraform

Interpolations may contain conditionals (if-else)

The syntax is:

```
CONDITION ? TRUEVAL : FALSEVAL
```

For example:

```
resource "aws_instance" "myinstance" {  
  [...]  
  count = "${var.env == "prod" ? 2 : 1 }"  
}
```

Conditions in Terraform

The support operators are:

- Equality: == and !=
- Numerical comparison: >, <, >=, <=
- Boolean logic: &&, ||, unary !

Conditions in Terraform

- Since both the instance and the SQS Queue are dependent upon the **S3 Bucket**.
- Terraform Waits until the bucket is created to begin creating the other two resources.

```
resource "aws_s3_bucket" "bucket1" {  
  
}  
  
resource "aws_instance" "instance1" {  
  ami = data.aws_ami.amazon_linux.id  
  instance_type = "t2.micro"  
  
  depends_on = [ aws_s3_bucket.bucket1 ]  
}  
  
resource "aws_sqs_queue" "queue1" {  
  
  depends_on = [ aws_s3_bucket.bucket1, aws_instance.instance1 ]  
}
```

Interpolation in Terraform

Interpolation

- In Terraform, you can Interpolate other values, Using `${...}`
- You can use simple math functions, refer to other variables, or use conditionals (if-else)
- I have been using them already throughout the course, without naming them
 - Variables: `${var.VARIABLE-NAME}` refers to a variable
 - Resources: `${aws_instance.name.id}` (type.resource-name.attr)
 - Data Source: `${data.template_file.name.rendered}` (data.type.resource- name.attr)

Interpolation: variables

Name	Syntax	Example
String variable	var.name	<code>\${var.SOMETHING}</code>
Map variable	var.MAP["key"]	1) <code>\${var.AMIS ["us-east-1"]}</code> 2) <code>2) \${lookup(var.AMIS, var.AWS_REGION)}</code>
List variable	var.LIST, var.LIST[i]	1) <code>\${var.subnets [i]}</code> 2) <code>\${join(",", var.subnets)}</code>

Interpolation: various

Name	Syntax	Example
Outputs of a module	module.Name.output	<code>\${module.aws_vpc.vpcid}</code>
Count information	count.FIELD	When using the attribute <code>count = number</code> in a resource, you can use <code>\${count.index}</code>
Path information	Path.TYPE	<code>path.cwd</code> (current directory) <code>path.module</code> (module path) <code>path.root</code> (root module path)
Meta information	terraform.FIELD	<code>terraform.env</code> shows active workspace

Interpolati on

Math:

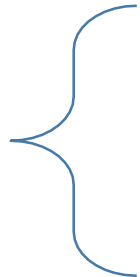
- Add (+), Subtract (-), multiply (*), and Divide (/) for float types
- Add (+), Subtract (-), multiply (*), Divide (/) and Modulo (%) for float types for integer types
- For example: $\{2+3*4\}$ results in 14

Iterations in Terraform

Method 1



Method 2



Resources

Metadata

✓ Meta-Arguments

- `depends_on`
- `count`
- `for_each`
- `provider`
- `lifecycle`

Resource Meta Arg: depends_on

Use the depends_on meta-argument to handle hidden resource or module dependencies that Terraform can't automatically infer.

Explicitly specifying a dependency is only necessary when a resource or module relies on some other resource's behavior but doesn't access any of that resource's data in its arguments.

Resource Meta Arg: depends_on – with resource

```
variable "storage_account_depends_on" {  
  # the value doesn't matter; we're just using this variable  
  # to propagate dependencies.  
  type    = any  
  default = []  
}  
  
resource "azurerm_storage_account" "test" {  
  name                        = "diagnostics${azurerm_resource_group.management.name}"  
  resource_group_name        = "${azurerm_resource_group.management.name}"  
  location                   = "${azurerm_resource_group.management.location}"  
  account_tier                = "Standard"  
  account_replication_type   = "LRS"  
  
  tags = {  
    environment = "diagnostics"  
  }  
  
  # This resource depends on whatever the variable  
  # depends on, indirectly. This is the same  
  # as using var.storage_account_depends_on in  
  # an expression above, but for situations where  
  # we don't actually need the value.  
  depends_on = [var.storage_account_depends_on]  
}
```

Resource Meta Arg: depends_on – with Module

```
module "diagnostic_logs" {  
  source = "../modules/diagnostic_logs"  
}  
  
module "storage_account" {  
  source = "../modules/storage_account"  
  
  storage_account_depends_on = [module.diagnostic_logs.logging]  
}
```

Then in your `diagnostic_logs` module you can configure indirect dependencies for the `logging` output to complete the dependency links between the modules:

```
output "logging" {  
  # Again, the value is not important because we're just  
  # using this for its dependencies.  
  value = {}  
  
  # Anything that refers to this output must wait until  
  # the actions for azurerm_monitor_diagnostic_setting.example  
  # to have completed first.  
  depends_on = [azurerm_monitor_diagnostic_setting.example]  
}
```

Resource Meta Arg: count

count is a meta-argument defined by the Terraform language. It can be used with modules and with every resource type.

The count meta-argument accepts a whole number, and creates that many instances of the resource or module. Each instance has a distinct infrastructure object associated with it, and each is separately created, updated, or destroyed when the configuration is applied.

```
resource "aws_instance" "server" {  
    count = 4 # create four similar EC2 instances  
  
    ami          = "ami-a1b2c3d4"  
    instance_type = "t2.micro"  
  
    tags = {  
        Name = "Server ${count.index}"  
    }  
}
```

Resource Meta Arg: for_each

for_each is a meta-argument defined by the Terraform language. It can be used with modules and with every resource type.

The for_each meta-argument accepts a map or a set of strings, and creates an instance for each item in that map or set. Each instance has a distinct infrastructure object associated with it, and each is separately created, updated, or destroyed when the configuration is applied.

```
resource "azurerm_resource_group" "rg" {  
  for_each = {  
    a_group = "eastus"  
    another_group = "westus2"  
  }  
  name      = each.key  
  location = each.value  
}
```

Resource Meta Arg: lifecycle

lifecycle is a nested block that can appear within a resource block. The lifecycle block and its contents are meta-arguments, available for all resource blocks regardless of type.

The following arguments can be used within a lifecycle block:

- create_before_destroy
- prevent_destroy
- ignore_changes

```
resource "azurerm_resource_group" "example" {  
  # ...  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

Operators in Terraform

Operators

```
graph LR; Operators[Operators] --- Arithmetic[Arithmetic Operators]; Operators --- Equality[Equality Operators]; Operators --- Comparison[Comparison Operators]; Operators --- Logical[Logical Operators];
```

Arithmetic Operators

Equality Operators

Comparison Operators

Logical Operators

Terraform: Equality Operators

The equality operators both take two values of any type and produce boolean values as results.

- `a == b` returns `true` if `a` and `b` both have the same type and the same value, or `false` otherwise.
- `a != b` is the opposite of `a == b`.

Terraform: Comparison Operators

The comparison operators all expect number values and produce boolean values as results.

- `a < b` returns `true` if `a` is less than `b`, or `false` otherwise.
- `a <= b` returns `true` if `a` is less than or equal to `b`, or `false` otherwise.
- `a > b` returns `true` if `a` is greater than `b`, or `false` otherwise.
- `a >= b` returns `true` if `a` is greater than or equal to `b`, or `false` otherwise.

Terraform: Logical Operators

The logical operators all expect bool values and produce bool values as results.

- `a || b` returns `true` if either `a` or `b` is `true`, or `false` if both are `false`.
- `a && b` returns `true` if both `a` and `b` are `true`, or `false` if either one is `false`.
- `!a` returns `true` if `a` is `false`, and `false` if `a` is `true`.

Terraform: Arithmetic Operators

The arithmetic operators all expect number values and produce number values as results:

- `a + b` returns the result of adding `a` and `b` together.
- `a - b` returns the result of subtracting `b` from `a`.
- `a * b` returns the result of multiplying `a` and `b`.
- `a / b` returns the result of dividing `a` by `b`.
- `a % b` returns the remainder of dividing `a` by `b`. This operator is generally useful only when used with whole numbers.
- `-a` returns the result of multiplying `a` by `-1`.

Terraform supports some other less-common numeric operations as [functions](#). For example, you can calculate exponents using the `pow` function.

Data Source in Terraform

S

- For certain providers (like AWS), terraform provides datasources

- Datasources provide you with dynamic information

A lot of data is available by AWS in a structured format using their API

Terraform also exposes this information using data sources

- Examples:

list of AMIs

List of availability Zones

- Another great example is the datasource that gives you all IP addresses in use by AWS
- This is great if you want to filter traffic based on an AWS region
 - e.g. allow all traffic from amazon instances in Europe
- Filtering traffic in AWS can be done using security groups Incoming and outgoing traffic can be filtered by protocol, IP range, and port
 - Similar to iptables (Linux) or a firewall

S

```
data "aws_ip_ranges" "european_ec2" {
  regions = [ "eu-west-1", "eu-central-1" ]
  services = [ "ec2" ]
}

resource "aws_security_group" "from_europe" {
  name = "from_europe"

  ingress {
    from_port = "443"
    to_port = "443"
    protocol = "tcp"
    cidr_blocks = [ "${data.aws_ip_ranges.european_ec2.cidr_blocks}" ]
  }
  tags {
    CreateDate = "${data.aws_ip_ranges.european_ec2.create_date}"
    SyncToken = "${data.aws_ip_ranges.european_ec2.sync_token}"
  }
}
```

nt

Output all the UBUNTU VM Image from Azure from one region but Use one of these image in Creating VM.

Deb ug

- TF_FORK=0
add an environment variable to prevent forking
#8795
- TF_LOG → TRACE, DEBUG, INFO, WARN or ERROR
- TF_LOG_PATH
- Found a bug? Report to the right place:
 - Check GitHub, it's highly likely a known bug/"feature"
 - <https://github.com/hashicorp/terraform> - Core Issues
 - <https://github.com/terraform-providers> - Provider Plugins
 - Check Golang SDK's bugs

Terraform tips

1. Use terraform console
 - a. `echo "random_string.new.result" | terraform console`
2. Use workspaces for simple scenarios
3. Isolate state files and don't use workspaces :)
4. To review output from terraform modules:
`terraform output -module=mymodule`
5. My state is changed every time when I'm running terraform with different users! (binary files/lambda functions)
 - a. `substr("${path.module}"/, length(path.cwd) + 1, -1)`
 - b. `ignore_changes = ["filename"]`

Terraform: common workflow issues

1. Mess up with workspaces
2. Hard-coded values
3. Not following naming convention (tags)
4. TF can't detect changes to computed values
5. Renaming modules, resources
6. Double references
7. Syntax problems
8. Variable "somevar" should be type map, got list
9. Timeouts
10. Permissions

Terraform: Sensitive information

1. terraform plan “-out plan-latest” - is not secured
2. terraform state - not secured.
 - a. Encryption on backend at rest
 - b. terraform pull - exposes sensitive
 - c. use data sources - grant only what you need
3. Data remote state - Not possible to expose just single or few outputs
4. Sensitive output
5. Encrypt tfvars
6. terraform output sensitive = true
 - a. seems ok? remote_secured = <sensitive>
 - b. terraform refresh → exposed, remote_secured = 79e6

Terraform: Sensitive information

1. How to handle secrets in state file?
 - a. Terrahelp - <https://github.com/opencredo/terrahelp>
 - b. Don't store secrets :) Use:
 - i. AWS/Google Cloud/Azure Key Vault/ etc. KMS -like + user-data mechanisms
 - ii. AWS System Manager Parameter store
 - iii. AWS Secrets manager
 - iv. Use resource Roles
 - v. If set master-password for DB service - change it after creation.
2. Secure state at rest using backend built-in encryption
3. Secure tfvars and other project/module specific information with:
 - a. pass - The password store - <https://www.passwordstore.org/>
 - b. git-crypt - <https://github.com/AGWA/git-crypt>

Overall Summary



Thank you

